

Enterprise DevOps Lab Platform — Complete Master Documentation

Author

Satish Mali

1. Objective

Build a complete enterprise-grade LOCAL DevOps lab environment focused on:

- Git
- GitHub
- Docker
- Kubernetes
- Jenkins
- Ansible
- Prometheus
- Grafana
- CI/CD Pipelines
- Monitoring & Observability
- DevSecOps

This roadmap is designed for:

- Real hands-on learning
 - Enterprise architecture understanding
 - GitHub portfolio preparation
 - Resume-ready projects
 - Kubernetes operational understanding
 - CI/CD implementation knowledge
 - Monitoring and DevSecOps exposure
-

2. IMPORTANT SCOPE

Primary Focus

The primary focus of this lab is:

- Docker
- Kubernetes
- Jenkins
- CI/CD

- Monitoring
- GitHub workflows
- Enterprise DevOps practices

Secondary / Optional Focus

These topics can be added later:

- Terraform
- Azure
- AWS
- GCP
- Multi-cloud

This avoids confusion between:

- Cloud Architect roadmap
- DevOps roadmap
- Kubernetes roadmap

3. FINAL ENTERPRISE FLOW

```
Developer Code
  ↓
Git & GitHub
  ↓
Docker Build
  ↓
Container Registry
  ↓
Jenkins CI/CD Pipeline
  ↓
Kubernetes Deployment
  ↓
Monitoring Stack
  ↓
Grafana Dashboards
  ↓
DevSecOps Security Validation
```

4. LOCAL LAB ARCHITECTURE

```
Windows 11 Laptop
  ↓
```

```
Docker Desktop
↓
kind Kubernetes Cluster
↓
kubectl Management
↓
Jenkins Container
↓
Prometheus Monitoring
↓
Grafana Visualization
```

5. REQUIRED SOFTWARE

Tool	Purpose
VS Code	Development IDE
Git	Version control
GitHub CLI	GitHub integration
Docker Desktop	Containers
kubectl	Kubernetes management
kind	Local Kubernetes cluster
Jenkins	CI/CD automation
Prometheus	Monitoring
Grafana	Dashboard visualization
Ansible	Automation

6. INSTALLATION PHASE

6.1 Install Git

```
winget install Git.Git
```

Verify:

```
git --version
```

6.2 Install GitHub CLI

```
winget install GitHub.cli
```

Verify:

```
gh --version
```

6.3 Login to GitHub CLI

```
& "C:\Program Files\GitHub CLI\gh.exe" auth login
```

Select:

- GitHub.com
- HTTPS
- Login with browser

Verify:

```
& "C:\Program Files\GitHub CLI\gh.exe" auth status
```

6.4 Configure Git

```
git config --global user.name "Satish Mali"  
git config --global user.email "YOUR_EMAIL"
```

Verify:

```
git config --global --list
```

7. CREATE ENTERPRISE PROJECT STRUCTURE

Create Main Folder

```
mkdir D:\enterprise-devops-platform  
cd D:\enterprise-devops-platform
```

Create Enterprise Folder Structure

```
mkdir DevOps  
mkdir Monitoring  
mkdir Security  
mkdir Documentation  
mkdir Screenshots
```

DevOps Structure

```
mkdir DevOps\Docker_Projects  
mkdir DevOps\Kubernetes  
mkdir DevOps\Jenkins_CICD  
mkdir DevOps\GitHub_Actions  
mkdir DevOps\Ansible  
mkdir DevOps\Microservices
```

Monitoring Structure

```
mkdir Monitoring\Prometheus  
mkdir Monitoring\Grafana  
mkdir Monitoring\AlertManager  
mkdir Monitoring\Dashboards
```

Security Structure

```
mkdir Security\Trivy  
mkdir Security\tfsec
```

```
mkdir Security\Checkov  
mkdir Security\SonarQube
```

8. GITHUB REPOSITORY CREATION

Initialize Git

```
git init
```

Create README

```
notepad README.md
```

Add:

```
# Enterprise DevOps Platform  
Enterprise-grade DevOps, Kubernetes, CI/CD, Monitoring and DevSecOps  
projects.
```

First Commit

```
git add .  
git commit -m "Initial enterprise devops platform structure"
```

Create GitHub Repository

```
& "C:\Program Files\GitHub CLI\gh.exe" repo create enterprise-devops-  
platform --public --source=. --remote=origin
```

Push Code

```
git branch -M main  
git push -u origin main
```

9. DOCKER PHASE

Objective

Learn:

- Docker images
- Containers
- Dockerfile
- Volumes
- Networks
- Port mapping
- Container lifecycle

10. PROJECT 1 — SIMPLE WEB APPLICATION

Create Folder

```
mkdir D:\enterprise-devops-platform\DevOps\docker_projects\simple-webapp
```

Structure

```
simple-webapp/  
├─ index.html  
├─ style.css  
├─ Dockerfile  
├─ deployment.yaml  
├─ service.yaml  
├─ Jenkinsfile  
└─ README.md
```

index.html

```
<!DOCTYPE html>  
<html>  
<head>  
<title>Enterprise DevOps Lab</title>  
<link rel="stylesheet" href="style.css">  
</head>
```

```
<body>
<h1>Enterprise DevOps Platform</h1>
<p>Docker + Kubernetes + Jenkins Project</p>
</body>
</html>
```

style.css

```
body {
  font-family: Arial;
  text-align: center;
  margin-top: 100px;
  background-color: #f5f5f5;
}
```

Dockerfile

```
FROM nginx:latest
COPY . /usr/share/nginx/html
EXPOSE 80
```

Build Docker Image

```
docker build -t enterprise-webapp .
```

Run Container

```
docker run -d -p 8080:80 enterprise-webapp
```

Verify

Open:

```
http://localhost:8080
```

11. IMPORTANT DOCKER CONCEPTS

Docker Port Mapping

Laptop Port 8080
↓
Container Port 80

Why Containers?

Containers provide:

- Lightweight runtime
- Fast deployment
- Isolation
- Scalability
- Portability
- Standardized environments

12. KUBERNETES PHASE

Objective

Learn:

- Pods
- Deployments
- Services
- ReplicaSets
- Scaling
- Self-healing
- Rolling updates
- NodePort
- kubectl operations

13. CREATE KIND CLUSTER

Create Cluster

```
kind create cluster --name ckd-lab
```

Verify

```
kubectl get nodes
```

14. LOAD IMAGE INTO KIND

```
kind load docker-image enterprise-webapp:latest --name cicd-lab
```

15. KUBERNETES DEPLOYMENT

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: enterprise-webapp

spec:
  replicas: 2

  selector:
    matchLabels:
      app: enterprise-webapp

  template:
    metadata:
      labels:
        app: enterprise-webapp

    spec:
      containers:
        - name: enterprise-webapp
          image: enterprise-webapp:latest
          imagePullPolicy: Never

      ports:
        - containerPort: 80
```

service.yaml

```
apiVersion: v1
kind: Service

metadata:
  name: enterprise-webapp-service

spec:
  type: NodePort

  selector:
    app: enterprise-webapp

  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Apply Deployment

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Verify

```
kubectl get pods
kubectl get svc
```

16. IMPORTANT KUBERNETES CONCEPTS

Docker vs Kubernetes

Docker

```
Single container runtime
```

Kubernetes

Container orchestration platform

Kubernetes provides:

- Auto healing
- Auto scaling
- Rolling updates
- Load balancing
- High availability
- Multi-container orchestration

17. JENKINS PHASE

Objective

Automate:

- GitHub pull
- Docker build
- Kubernetes deployment
- CI/CD pipelines

Run Jenkins

```
docker run -d --name jenkins -p 8081:8080 -p 50000:50000 jenkins/jenkins:lts
```

Access Jenkins

```
http://localhost:8081
```

18. JENKINS PIPELINE FLOW

```
GitHub Push
  ↓
Jenkins Trigger
  ↓
```

```
Docker Build
  ↓
Kubernetes Deployment
```

19. SAMPLE JENKINSFILE

```
pipeline {
  agent any

  stages {
    stage('Build Docker Image') {
      steps {
        bat 'docker build -t enterprise-webapp .'
      }
    }

    stage('Deploy to Kubernetes') {
      steps {
        bat 'kubectl apply -f deployment.yaml'
        bat 'kubectl apply -f service.yaml'
      }
    }
  }
}
```

20. ANSIBLE PHASE

Objective

Automate:

- Linux configuration
- Package installation
- Docker installation
- Kubernetes prerequisites
- Configuration management

Ansible Structure

```
Ansible/
├─ inventory.ini
```

```
|─ ansible.cfg
|─ playbooks/
|─ roles/
└─ README.md
```

Sample Playbook

```
- hosts: all
  become: yes

  tasks:

    - name: Install nginx
      apt:
        name: nginx
        state: present
```

Run Playbook

```
ansible-playbook -i inventory.ini playbooks/install-nginx.yml
```

21. PROMETHEUS + GRAFANA PHASE

Objective

Build enterprise monitoring platform for:

- Kubernetes
- Docker
- Linux
- Jenkins
- Containers

22. MONITORING FLOW

```
Applications
  ↓
Prometheus
  ↓
```

Grafana
↓
Alerts

23. WHAT PROMETHEUS DOES

Prometheus continuously collects:

- CPU metrics
- Memory metrics
- Container metrics
- Pod metrics
- Network traffic
- Disk usage

Prometheus acts as:

Monitoring Engine + Time-Series Database

24. WHAT GRAFANA DOES

Grafana provides:

- Dashboards
- Visual graphs
- Monitoring panels
- SLA dashboards
- Alerting
- Infrastructure reports

25. PROMETHEUS + GRAFANA USING DOCKER COMPOSE

docker-compose.yml

```
version: '3'

services:

  prometheus:
```

```
image: prom/prometheus
ports:
  - "9090:9090"

grafana:
  image: grafana/grafana
  ports:
    - "3000:3000"
```

Start Monitoring Stack

```
docker compose up -d
```

Access

Prometheus:

```
http://localhost:9090
```

Grafana:

```
http://localhost:3000
```

26. DEVSECOPS PHASE

Security Tools

Tool	Purpose
Trivy	Container scanning
tfsec	Terraform scanning
Checkov	IaC security
SonarQube	Code quality
Snyk	Dependency scanning

27. SECURITY PIPELINE

```
Code Commit
  ↓
Security Scan
  ↓
Build Validation
  ↓
Deployment
```

28. TROUBLESHOOTING SECTION

Docker Container Not Running

Check:

```
docker ps -a
```

View logs:

```
docker logs CONTAINER_ID
```

Kubernetes Pods Not Starting

Check:

```
kubectl get pods
```

Describe pod:

```
kubectl describe pod POD_NAME
```

View logs:

```
kubectl logs POD_NAME
```

kind Cluster Missing

Verify:

```
kind get clusters
```

Recreate:

```
kind create cluster --name cicd-lab
```

Jenkins Not Opening

Verify:

```
docker ps
```

Restart:

```
docker restart jenkins
```

29. FINAL ENTERPRISE PROJECTS

Project 1

Docker Website Deployment

Project 2

Kubernetes Multi-Pod Deployment

Project 3

Jenkins CI/CD Pipeline

Project 4

Prometheus + Grafana Monitoring Stack

Project 5

DevSecOps Security Pipeline

30. FINAL RESUME PROJECT

Enterprise Kubernetes CI/CD Platform

Implemented:

- Docker containerization
 - Kubernetes orchestration
 - Jenkins CI/CD automation
 - Prometheus monitoring
 - Grafana dashboards
 - DevSecOps security scanning
-

31. FINAL LEARNING ROADMAP

Phase	Duration
Git + Docker	1–2 Months
Kubernetes	2–3 Months
Jenkins CI/CD	2 Months
Monitoring	1–2 Months
DevSecOps	1 Month
Enterprise Projects	2 Months

32. FINAL OUTCOME

After completing this roadmap, you will have:

- Strong GitHub portfolio
- Real DevOps projects
- Kubernetes operational understanding
- CI/CD implementation knowledge
- Monitoring expertise
- DevSecOps exposure
- Enterprise-level documentation
- Interview-ready architecture understanding